

4/9/26 Natural Language Processing (NLP) 20/9/26

Natural language $\rightarrow$ Human language

NLP Applications:-

Contextual Advertisements, Search Engines, Email filtering etc.

Common NLP Tasks:-

Text/doc classification
Sentiment Analysis
Info retrieval
Parts of Speech tagging
Knowledge graphing

Approaches to NLP:-

1) Heuristic Approach:- Naive approach.

Ex: Regular Expressions
WordNet
Open Mind Common Sense

(Picking up from database)

(Rules are written manually)

2) Machine learning:-

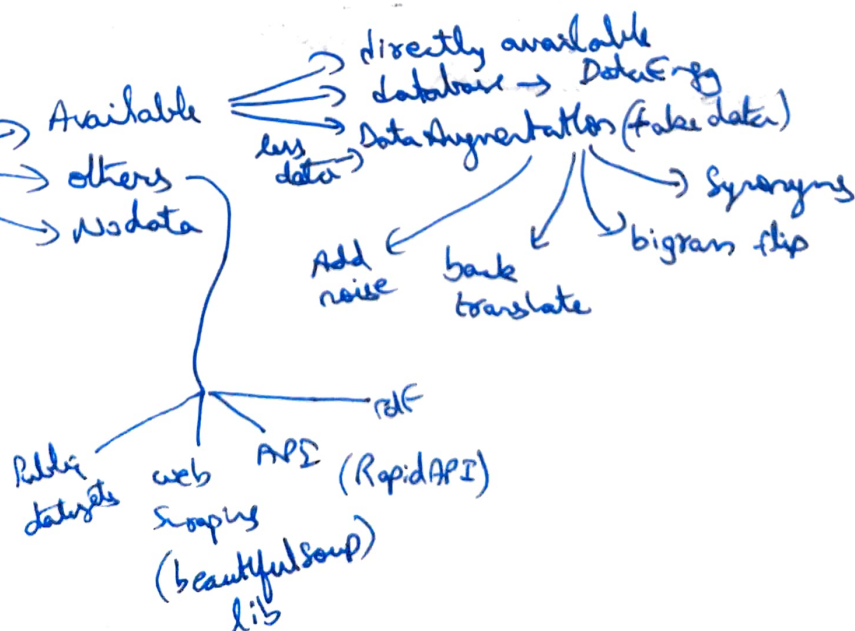
Algorithms are made for handling textual data

3) Deep learning Approach:-
Seq data

NLP Pipeline:-

- i) Data Acquisition
- ii) Text preparation
- iii) Feature Engineering
- iv) Modelling
- v) Deployment

Data Acquisition:-



Text Preparation:-

cleaning

- html tag cleaning
- emoji
- Spelling check (TextBlob lib)

basic preprocessing



optional

- stop word removal
- stemming
- removing digits
- lowercase

Advance

Preprocessing

- POS tagging
- Parsing
- coreference resolution

Feature Engineering:-

Text → numbers.

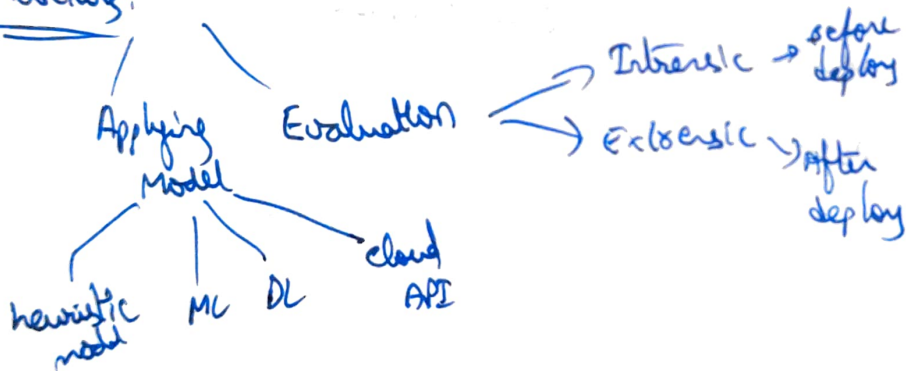
Bag of words

TF-IDF

OHE

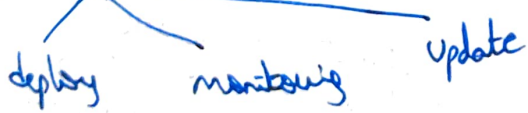
word2vec

Modelling:-



Perplexity → model eval

Deployment:-



Text Preprocessing:-

Lower casing:-

```
text = "Hi"  
text.lower()
```

If a col review with upper & lower to change all
`df['review'].str.lower()`

To remove html tags:-

```
import re  
Pattern = re.compile('<.*?>')  
Pattern.sub('', text)  
df['review'] = df['review'].apply(remove_html)
```

To remove links:-

```
re.compile(r'https?://\S+/www.\S+')
```

To remove punctuation:-

```
import string  
Punc = string.punctuation  
for char in Punc:  
    text = text.replace(char, '') } (or) text.translate(str.maketrans('','',punc))
```

chat word treatment:-

lmao, Grn, ASAP...

Spelling mistake:-

```
from textblob import TextBlob  
tb = TextBlob(text)  
tb.correct().string()
```

Removing stop words:-

a, an, the, is, my ----

```
from nltk.corpus import stopwords  
stopwords = stopwords.words('english')
```

Emojis:-

```
import emoji  
Pout(emoji.emojize(text))
```

Tokenization:-

breaking text to small parts known as tokens.

using split:-

text.split(' ') // word wise

text.split('.') // sentence wise

using re:-

re.compile('[.!?]').split(text)

using nlTK:-

from nltk.tokenize import word_tokenize, sent_tokenize.

using Spacy:-

import Spacy

nlp = spacy.load('en-core-web-sm')

doc1 = nlp(text).

Stemming:- Reduces inflections in words to their root word.

↳ modification of word to express differently
walk → walking, walks, walked ~

from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

ps.stem(word)

Lemmatization:- same as stemming but more accurate & slow.

from nltk.stem import WordNetLemmatizer

wl = WordNetLemmatizer()

wl.lemmatize(word, pos='v')

Text Representation:-

Text → numbers

Techniques:-

One Hot Encoding

Bag of Words

Ngrams

TF-IDF

word2vec

} ML

} DL

Corpus:- All the words in the dataset's combination /
(c) No of ~~unique~~ words in whole dataset Combined

Vocabulary: Unique words in Corpus.
(v)

Document(d):- Each cell value.

OneHot Encoding:-

Each word in document will be represented with size of length of vocabulary).

D1 - People watch me

D2 - me watch you

vocabulary - People watch me you
 1 0 0 0 → People
 0 1 0 0 → watch
 0 0 1 0 → me
 0 0 0 1 → you.

D₁ → [[1 0 0 0] [0 1 0 0] [0 0 1 0]]

Pros:-
 Intuitive
 Easy to implement

Cons:-
 > Sparsity
 > overfitting
 > Issues during Prediction
 (OOV - out of vocabulary)
 > No Semantic meaning capturing

Bag of Words:-

Used in Text classification.

D1 → People watch campus - 1

D2 → Campus watch campus - 1

D3 → People write comment - 1

D4 → Campus write comment - 1

	People	watch	campus	write	comment
D1	1	1	1	0	0
D2	0	1	2	0	0
D3	1	0	0	1	1
D4	0	0	1	1	1

No. of times each word is repeated in each document.

Main Intuition is in each category of words the frequency will be same.

order & context is not a matter, frequency matters

from sklearn.feature-extraction.text import CountVectorizer

cv = CountVectorizer()

bow = cv.fit_transform(df['text'])

To see vocabulary → (cv.vocabulary_)

Advantages

Simple & Intuitive
Handles OOV
Semantic meaning

disadvantages

- Sparsity
- oov loses info
- order of words

Ex: This is a good movie
This is not a good movie
semantically they are
complete opposites

but



not
much
difference

N-grams:-

D1 People watch campus
D2 Campus watch campus
D3 People write comment
D4 Campus write comment

Bigram

	People watch	watch campus	campus watch	People write	write comment	comment
D1	1	0	0	0	0	0
D2	0	1	1	0	0	0
D3	0	0	0	1	1	0
D4	0	0	0	0	0	1

Trigram

	People watch campus	campus watch campus	People write comment	comment
D1	0	0	0	0
D2	0	1	0	0
D3	0	0	1	0
D4	0	0	0	1

quadgram is not possible

Bag of words = Unigram

cv = Count Vectorizer (ngram_range = (3,3)) → Trigram
(1,2) → unigram, bigram
(1,3) → uni, bi, tri.

This movie is very good
 This movie is not good

this movie	movie is	as very	very good	is not	not good
1	1	1	1	0	0
0	0	0	0	1	1

Advantages

- captures semantic meaning
- Easy to implement

Disadv.

slows down algo
 OOV is not handled but ignored

TF-IDF

Term frequency - Inverse Document frequency.

Each word will be assigned with certain weightage.

$$TF(t, d) = \frac{\text{No of occ of term } t \text{ in doc } d}{\text{Total no of terms in doc } d}$$

$$IDF(t) = \log_e \left(\frac{\text{Total no of docs in the corpus}}{\text{No of docs with term } t \text{ in them} + 1} \right)$$

log is used so, that if enc 10000 then IDF dominates over TF

$$TF(\text{corpus}, D_1) = \frac{1}{3}$$

$$IDF(\text{corpus}) = \log\left(\frac{4}{3}\right)$$

from sklearn.feature-extraction.text

tfidf = TfidfVectorizer()

tfidf.fit_transform(df['sent'])

input TfidfVectorizer

Advantages

Information Retrieval

Disadvantages

Sparsity

OOV

More dim

No semantic meaning capturing

Word2Vec:- Prediction based.

captures semantic meaning
low dimension vectors & mostly non zero which
avoids overfitting.

Can be used in 2 ways $\Rightarrow$ Pretrained (300 dim vectors of 3 million words)
 $\Rightarrow$ Self-train. (google news dataset)

Uses cosine similarity.

word2vec creates features depending on Vocabulary.

	king	queen	man	woman	monkey
gender	1	0	0	0	0
wealth	1	1	0.3	0.3	0
Power	1	0.7	0.2	0.2	0
weight	0.8	0.4	0.6	0.5	0.1

like these king $\rightarrow [1, 1, 1, 0.8]$

features are generated.
(Neural Networks are used to create features)
& we can't know what type of feature it is.

2 words sharing meaning and also similar content also share similar vector representations.

Types of word2vec

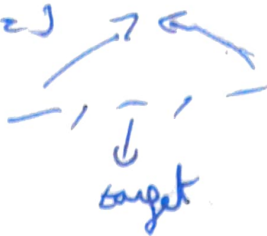
CBOW
(Contexts
Bag of Words)

skip-gram

CBOW +

watch context for data science context

Suppose window size = 2

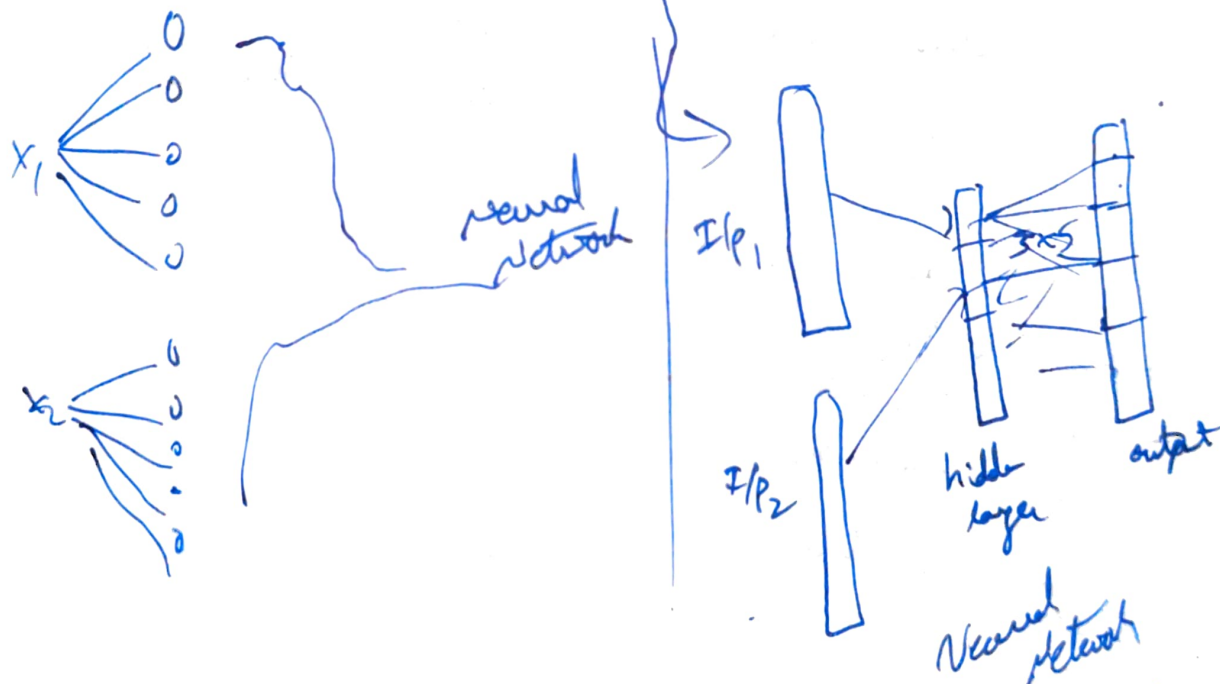


X
watch, context
for data science

Y
science
for data

watch
context
for
data
science

$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$



so the embeddings will be 3 dim weights from hidden layer to o/p layer of each word.

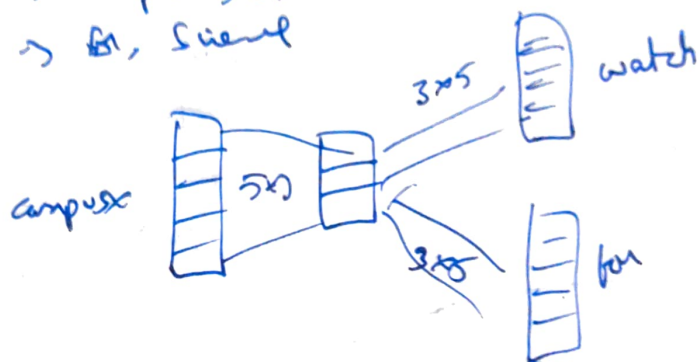
Train the data & adjust the weights & minimize the loss.

Skip gram

Reverse of CBOW.

given Predicted words we need to get content words

campus → watch, for
for → campus, data
data → for, student



Whenever, working with small data → CBOW helps
" " " large data → skip-gram helps

To improve word2vec quality

- > Increase training data.
- > Increase dimension of vectors.
- > Increase window size.

Training with own data

import gensim

from nltk import sent_tokenize

from gensim.utils import simple_preprocess

story = []

raw_sent = sent_tokenize(corpus)

for row in raw_sent:

story.append(simple_preprocess(row))

model = gensim.models.word2vec(window=10, min_count=2,
model.build_vocab(story) workers=4)

model.train(story, total_examples=model.corpus_count,
epochs=model.epochs)

model.wv.most_similar('ringsnow')

model.wv['king'].

Text classification:-
 Ex: Email spam classifier, language detection, fake news detection
 Sentiment Analysis

Types

- binary classification → 2 classes
- Multiclass " " → 2 or more classes
- Multilabel " "

Approaches:-

Heuristic approaches
 ↓
 Making some judged approaches

APIS
 ↓
 Using APIs which takes direct text I/p & does classification

ML

- Bow
- TFIDF

DL

- Naive Bayes
- Random Forest
- RNN
- LSTM
- BERT

Try using ensemble techniques, Heuristic features for better results. Avoid Imbalance datasets

Parts-of-Speech tagging (Pre-processing step)
 Labelling each word in a sentence with its parts of speech

Applications -

- Named-Entity Recognition.
- Question-Answer System
- Divide the words of same with different meaning.

```
!pip install spacy
import spacy
nlp = spacy.load('en-core-web-sm')
doc = nlp('I will eat you')
doc[0].pos_ // Pronoun [PRON] Coarse grained POS
doc[0].tag_ // Present Participle [PRP] Fine grained POS
doc[0].tag_ // It goes fine & assign finer way.
spacy.explain('PRP') // Present Participle.
```

Viterbi & Hidden Markov Model algos are used for POS tagging.

Advanced features

Sachin is ¹⁰a great batsman

tokens $\rightarrow 5$

stop words $\rightarrow 2$

words $\rightarrow 3$ (Tokens - stop words)

i) CWC-min:

$z_1 \rightarrow$

$z_2 \rightarrow$

$$\frac{\# \text{ common words}}{\min(\text{words}(z_1, z_2))}$$

ii) CWC-max:

$$\frac{\# \text{ common words}}{\max(\text{words}(z_1, z_2))}$$

iii) CSC-min =

$$\frac{\# \text{ common stop words}}{\min(\text{stop words}(z_1, z_2))}$$

iv) CSC-max =

$$\frac{\# \text{ common stop words}}{\max(\text{stop words}(z_1, z_2))}$$

v) CTC-min =

$$\frac{\# \text{ common tokens}}{\min(\text{tokens}(z_1, z_2))}$$

vi) CTC-max =

$$\frac{\# \text{ common tokens}}{\max(\text{tokens}(z_1, z_2))}$$

vii) last-word-eq $\Rightarrow$ 1 if same last word
0 if not same last word

viii) first-word-eq $\rightarrow$ 1 if same first word
0 if not

length based features

i) mean-len $\Rightarrow \frac{10+15}{2}$ $z_1 \rightarrow 10$ tokens
 $z_2 \rightarrow 15$ tokens

ii) abs-len-diff $\Rightarrow |10-15| = 5$

iii) longest-substr-ratio $\Rightarrow \frac{\log \text{ common substr}}{\min(\text{words}(z_1, z_2))}$